

MASTERING DESIGN PATTERNS *IN C++*

A COMPREHENSIVE GUIDE TO
MODERN SOFTWARE DEVELOPMENT



WRITTEN BY

RAY

sderay.com

Connect with SDE Ray

Stay connected with **SDE Ray**, the author of *Mastering Design Patterns in C++*. Follow me on these platforms to explore insights into software development, design patterns, and the art of clean code. Engage with my posts, share your thoughts, and join the conversation!

Website

Visit my official website for detailed blogs, tutorials, and resources on software development:
sderay.com

X

Follow me on X (formerly Twitter) for quick insights, coding tips, and updates: [@sde_ray](https://twitter.com/sde_ray)

Instagram

Join me on Instagram for a behind-the-scenes look at my coding journey, snapshots of my work, and more: [@sde.ray](https://www.instagram.com/sde.ray)

Please follow and support my work!

Table of Contents

1. Introduction

2. Understanding Design Patterns

3. Creational Patterns

- Singleton Pattern
- Prototype Pattern
- Factory Method Pattern
- Abstract Factory Pattern
- Builder Pattern

4. Structural Patterns

- Adapter Pattern
- Composite Pattern
- Decorator Pattern
- Facade Pattern
- Flyweight Pattern
- Proxy Pattern

5. Behavioral Patterns

- Chain of Responsibility Pattern
- Command Pattern
- Iterator Pattern
- Mediator Pattern
- Memento Pattern
- Observer Pattern
- State Pattern
- Strategy Pattern
- Template Method Pattern
- Visitor Pattern

6. Conclusion

Mastering Design Patterns in C++

Introduction

In the world of software development, writing code that is scalable, maintainable, and efficient is not just a goal—it is a necessity. Design patterns offer proven solutions to common problems, acting as blueprints for building robust software. This book, *Mastering Design Patterns in C++*, is your complete guide to understanding, implementing, and leveraging design patterns in modern C++.

Whether you're building a simple application or architecting a complex system, design patterns can help streamline your development process. This book explores the foundational principles of design patterns, grouped into three key categories: **Creational**, **Structural**, and **Behavioral**. Each chapter dives deep into individual patterns, offering clear explanations, detailed examples, and practical scenarios.

What You'll Learn:

- The significance and structure of design patterns.
- How to implement patterns such as Singleton, Factory, Builder, and Observer in C++.
- Best practices for making your code more reusable, flexible, and adaptable.
- How design patterns can solve real-world challenges in software development.

With this book as your guide, you will not only master the implementation of design patterns but also understand when and why to use them, empowering you to write elegant, high-performance C++ code.

Note:

Some additional behavioral patterns will be added in future versions to further enrich your understanding and capabilities.

Design Patterns in C++

What Are Design Patterns?

Design patterns are tried-and-tested solutions to common problems that developers face when building software. Think of them as best practices or templates that you can follow to solve specific coding challenges. By using design patterns, you can write code that's easier to understand, maintain, and reuse.

At their core, design patterns are reusable templates that empower developers to address recurring challenges in software design. They offer a structured approach, promoting adaptability for specific design problems.

Types of Design Patterns

Design patterns are grouped into three main categories based on what they focus on:

1. Creational Patterns: How Objects Are Created

These patterns are all about making sure objects are created in the best possible way for your situation. Instead of just using `new` to create an object, creational patterns give you more control over the creation process. They help you manage how objects are made, ensuring your code is flexible and can easily adapt to changes.

2. Structural Patterns: How Objects Are Organized

Structural patterns focus on how objects and classes fit together to form larger structures. These patterns are about making sure that if one part of your system changes, the rest doesn't need to. They help you compose objects and classes in a way that makes your system more flexible and easier to understand.

3. Behavioral Patterns: How Objects Communicate

Behavioral patterns deal with how objects interact and communicate with each other. These patterns help define clear roles and responsibilities for objects, making sure your system behaves correctly and is easy to manage and extend.

Explore Individual Design Patterns

Each design pattern solves a specific problem in a certain way. Below are the types of patterns, with links to detailed explanations and examples of each:

Creational Patterns

- **Singleton Pattern:** Ensures there's only one instance of a class and provides a global access point to it.
- **Prototype Pattern:** Allows you to create new objects by copying an existing object (a prototype).
- **Factory Method Pattern:** Lets you create objects without specifying the exact class of the object that will be created.
- **Abstract Factory Pattern:** Lets you create families of related objects without specifying their concrete classes.
- **Builder Pattern:** Helps you construct complex objects step by step, separating the construction process from the final representation.

Structural Patterns

- **Adapter Pattern:** Allows two incompatible interfaces to work together.
- **Composite Pattern:** Lets you compose objects into tree structures to represent part-whole hierarchies.
- **Decorator Pattern:** Adds new responsibilities to an object dynamically, without altering its structure.
- **Facade Pattern:** Provides a simple interface to a complex system, making it easier to use.
- **Flyweight Pattern:** Reduces memory usage by sharing as much data as possible with similar objects.
- **Proxy Pattern:** Provides a placeholder or proxy for another object to control access to it.

Behavioral Patterns

- **Chain of Responsibility Pattern:** Passes a request along a chain of handlers, where each handler decides whether to process the request or pass it along.
- **Command Pattern:** Encapsulates a request as an object, allowing you to parameterize and queue requests.
- **Iterator Pattern:** Provides a way to access the elements of a collection sequentially without exposing its underlying representation.
- **Mediator Pattern:** Defines an object that manages communication between other objects, promoting loose coupling.
- **Memento Pattern:** Captures and restores an object's internal state without breaking encapsulation.
- **Observer Pattern:** Notifies all dependent objects when an object's state changes, automatically updating them.
- **State Pattern:** Allows an object to alter its behavior when its internal state changes.
- **Strategy Pattern:** Defines a family of algorithms, encapsulates each one, and makes them interchangeable.
- **Template Method Pattern:** Defines the steps of an algorithm, allowing subclasses to provide their own implementation for some steps.
- **Visitor Pattern:** Let you add further operations to objects without modifying them.

Why Use Design Patterns?

- **Consistency:** Ensures that your code follows best practices.
- **Reusability:** Patterns are designed to solve common problems, so you can reuse them in different parts of your application.

- **Scalability:** Helps your code adapt to changes and grow as your project evolves.

Contributing

I welcome contributions! If you have a new pattern, improvement, or a different approach to an existing pattern, feel free to email me at sderay.mail@gmail.com with the subject **Design Pattern Suggestion** or by tagging me on X(twitter): [@sde_ray](https://twitter.com/sde_ray). Let's build the most comprehensive C++ design patterns resource together!

Singleton Design Pattern

Introduction

The Singleton Design Pattern is a creational pattern that ensures a class has only one instance while providing a global point of access to that instance. This pattern is widely used when a single object is needed to coordinate actions across the entire system, such as managing a database connection, logging system, or configuration settings.

Key Concepts

- **Single Instance:** The class ensures that only one instance is created, preventing the creation of multiple instances.
- **Global Access:** The Singleton instance is accessible globally, allowing various parts of the application to interact with it.
- **Private Constructor:** The constructor is private or protected to prevent direct instantiation from outside the class.

When to Use the Singleton Pattern

- When you need to ensure there is exactly one instance of a class.
- When the single instance should be accessible globally across different parts of the application.
- When you want to control access to a shared resource, such as logging, configuration, or database connections.

Example in C++

Implementation Overview

Below is a simple implementation of the Singleton Design Pattern in C++.

```
#include <iostream>
#include <mutex>

class Singleton {
private:
    static Singleton* instance; // Static instance pointer
    static std::mutex mtx;      // Mutex for thread safety

    // Private constructor to prevent direct instantiation
    Singleton() {
        std::cout << "Singleton instance created." << std::endl;
    }

    // Delete the copy constructor and assignment operator
    Singleton(const Singleton&) = delete;
    Singleton& operator=(const Singleton&) = delete;

public:
    // Static method to get the single instance of the class
    static Singleton* getInstance() {
        std::lock_guard<std::mutex> lock(mtx); // Ensure thread safety
        if (instance == nullptr) {
            instance = new Singleton();
        }
        return instance;
    }

    void showMessage() {
        std::cout << "Hello from Singleton!" << std::endl;
    }
};

// Initialize the static members
Singleton* Singleton::instance = nullptr;
std::mutex Singleton::mtx;

int main() {
    // Get the singleton instance and use it
    Singleton* singleton1 = Singleton::getInstance();
    singleton1->showMessage();

    // Try to get another instance and use it
    Singleton* singleton2 = Singleton::getInstance();
    singleton2->showMessage();

    // Check if both instances are the same
    if (singleton1 == singleton2) {
        std::cout << "Both instances are the same." << std::endl;
    }
}
```

```
    return 0;  
}
```

How It Works

- **Private Constructor:** The constructor is private, preventing other classes from creating instances directly.
- **Static Instance:** A static pointer (`instance`) holds the single instance of the class.
- **Thread Safety:** The `std::mutex` and `std::lock_guard` ensure that only one thread can create the instance at a time, preventing multiple instances in a multithreaded environment.
- **No Copying:** The copy constructor and assignment operator are deleted to prevent copying the singleton instance.

Benefits of the Singleton Pattern

- **Controlled Access:** The pattern controls how and when the instance is created and accessed.
- **Memory Efficiency:** Since only one instance is created, memory usage is optimized.
- **Consistency:** A single instance ensures consistent behavior across the application.

Common Pitfalls

- **Global State:** Excessive use of the Singleton pattern can lead to hidden dependencies and make the codebase harder to understand and test.
- **Thread Safety:** In multithreaded environments, you must ensure that the Singleton instance is created safely. This example uses a mutex for thread safety.

Conclusion

The Singleton Design Pattern is a powerful tool when you need a single, globally accessible instance of a class. It provides both control and consistency in object creation, making it ideal for managing shared resources like configurations, logging systems, or database connections.

Prototype Design Pattern

Introduction

The Prototype design pattern is a **creational pattern** that allows you to create new objects by copying or cloning an existing object, known as the prototype. This pattern is useful when creating objects is resource-intensive, and you want to avoid the overhead of initializing new objects from scratch. Instead of creating objects from scratch, this pattern creates similar objects by cloning existing ones. The clone performs a Deep Copy, hence any changes made to the clone object doesn't change the value of original object.

When to Use the Prototype Pattern

- When object creation is costly (in terms of time or resources).
- When the system should be independent of how its objects are created, composed, and represented.
- When you need to create objects that are similar but not exactly the same.

Structure of the Prototype Pattern

The pattern typically involves the following components:

1. **Prototype Interface:** Declares a cloning method.
2. **Concrete Prototype:** Implements the cloning method. This class defines the object that will be copied.
3. **Client:** Creates a new object by requesting a clone from the Prototype.

Example in C++

Here's a simple example of how you might implement the Prototype pattern in C++:

```
#include <iostream>
#include <unordered_map>

// Prototype Interface
class Shape {
public:
    virtual Shape* clone() const = 0;
    virtual void draw() const = 0;
    virtual void modify(int newValue1, int newValue2 = 0) = 0;
    virtual ~Shape() {}
};

// Concrete Prototype: Circle
class Circle : public Shape {
private:
    int radius;
public:
    Circle(int r) : radius(r) {}
    Shape* clone() const override {
        return new Circle(*this); // Deep copy of the existing object
    }
    void draw() const override {
        std::cout << "Drawing a Circle with radius: " << radius << std::endl;
    }
    void modify(int newRadius, int) override {
        radius = newRadius;
    }
};

// Concrete Prototype: Rectangle
class Rectangle : public Shape {
private:
    int width, height;
public:
    Rectangle(int w, int h) : width(w), height(h) {}
    Shape* clone() const override {
        return new Rectangle(*this); // Deep copy of the existing object
    }
    void draw() const override {
        std::cout << "Drawing a Rectangle with width: " << width
                  << " and height: " << height << std::endl;
    }
    void modify(int newWidth, int newHeight) override {
        width = newWidth;
        height = newHeight;
    }
};

// Client
class PrototypeFactory {
private:
    std::unordered_map<std::string, Shape*> prototypes;
public:
```

```

PrototypeFactory() {
    prototypes["Circle"] = new Circle(10);
    prototypes["Rectangle"] = new Rectangle(5, 7);
}

Shape* createShape(const std::string& type) {
    return prototypes[type]->clone();
}

~PrototypeFactory() {
    // Delete all prototypes to avoid memory leaks
    for (auto& pair : prototypes) {
        delete pair.second;
    }
}

};

int main() {
    PrototypeFactory factory;

    // Create a clone of the Circle and modify the original
    Shape* circle = factory.createShape("Circle");
    Shape* circleClone = circle->clone();

    std::cout << "Before modifying the original Circle:" << std::endl;
    circle->draw();
    circleClone->draw();

    // Modify the original Circle
    circle->modify(20);

    std::cout << "\nAfter modifying the original Circle:" << std::endl;
    circle->draw();
    circleClone->draw();

    // Create a clone of the Rectangle and modify the original
    Shape* rectangle = factory.createShape("Rectangle");
    Shape* rectangleClone = rectangle->clone();

    std::cout << "\nBefore modifying the original Rectangle:" << std::endl;
    rectangle->draw();
    rectangleClone->draw();

    // Modify the original Rectangle
    rectangle->modify(15, 10);

    std::cout << "\nAfter modifying the original Rectangle:" << std::endl;
    rectangle->draw();
    rectangleClone->draw();

    // Clean up manually to avoid memory leaks
    delete circle;
    delete circleClone;
    delete rectangle;
    delete rectangleClone;

    return 0;
}

```

Explanation

- **Prototype Interface (Shape)**: This abstract class declares the `clone()` method that returns a `Shape*`, allowing objects to be cloned.
- **Concrete Prototypes (Circle and Rectangle)**: These classes implement the `clone()` method, enabling them to be copied.
- **Prototype Factory (PrototypeFactory)**: The factory stores a collection of prototype objects. The `createShape()` method clones a prototype and returns the copy to the client.
- **Main Function**: Demonstrates how to create new objects using the prototype pattern by cloning existing shapes.

Key Points

- Cloning an object is often faster than creating a new one from scratch.
- The Prototype pattern allows for the flexibility to add or modify prototypes without changing the code that uses them.
- You can enhance the Prototype pattern by using deep or shallow copies, depending on the requirements.

This pattern is particularly useful when dealing with complex objects that need to be instantiated multiple times with similar configurations.

Factory Design Pattern

Introduction

The Factory Design Pattern is a **creational design pattern** that provides a way to create objects without directly specifying their exact class. Instead, you use a factory that knows how to create different types of objects. It's useful when you have a common interface or base class and you want to delegate the responsibility of instantiating concrete classes to a factory.

Key Concepts

- **Factory Method:** A method in the factory class that decides which concrete class to instantiate based on input parameters.
- **Concrete Product:** The specific implementation of the product that the factory creates.
- **Product Interface:** A common interface or abstract class that all products share.

When to Use the Factory Pattern

- When you want to centralize the creation of objects to ensure consistency and control.
- When you have a common interface or base class, and the exact type of the object isn't known until runtime.
- When the creation process involves complex logic that should not be exposed to the client.

Example in C++

Code Overview

Below is an example of a simple factory that creates different types of `Shape` objects like `Circle`, `Rectangle`, and `Triangles`.

```
#include <iostream>

// Interface and should only contain all the pure virtual functions.
class Shape {
public:
    virtual void draw() const = 0;
    virtual ~Shape() {}
};

// Concrete Product: Circle
class Circle : public Shape {
public:
    void draw() const override {
        std::cout << "Drawing a Circle" << std::endl;
    }
};

// Concrete Product: Rectangle
class Rectangle : public Shape {
public:
    void draw() const override {
        std::cout << "Drawing a Rectangle" << std::endl;
    }
};

// Concrete Product: Triangle
class Triangle : public Shape {
public:
    void draw() const override {
        std::cout << "Drawing a Triangle" << std::endl;
    }
};

// Factory Class. Note that this class can provide some default implementation of the factory method too. All functions need not be pure virtual
class ShapeFactory {
public:
    enum ShapeType {
        CIRCLE,
        RECTANGLE,
        TRIANGLE
    };

    static Shape* createShape(ShapeType type) {
        switch (type) {
            case CIRCLE:
                return new Circle();
            case RECTANGLE:
                return new Rectangle();
            case TRIANGLE:
                return new Triangle();
            default:
                return nullptr;
        }
    }
};
```

```

        return nullptr;
    }
}

};

int main() {
    ShapeFactory::ShapeType shapeType;

    // Create and use a Circle
    shapeType = ShapeFactory::CIRCLE;
    Shape* shape1 = ShapeFactory::createShape(shapeType);
    shape1->draw();
    delete shape1;

    // Create and use a Rectangle
    shapeType = ShapeFactory::RECTANGLE;
    Shape* shape2 = ShapeFactory::createShape(shapeType);
    shape2->draw();
    delete shape2;

    // Create and use a Triangle
    shapeType = ShapeFactory::TRIANGLE;
    Shape* shape3 = ShapeFactory::createShape(shapeType);
    shape3->draw();
    delete shape3;

    return 0;
}

```

Explanation

- **Product Interface:** The `Shape` class defines the interface for all shape objects.
- **Concrete Products:** `Circle`, `Rectangle`, and `Triangle` implement the `Shape` interface and provide specific behaviors for each shape.
- **Factory Class:** The `ShapeFactory` class contains a method `createShape` that takes a `ShapeType` enum and returns a new instance of the corresponding shape.
- **Client Code:** The `main` function shows how to use the factory to create different shapes without knowing the details of how they are created.

Benefits of the Factory Pattern

- **Encapsulation:** The object creation logic is centralized in the factory, making the code easier to manage and modify.
- **Flexibility:** New product types can be added with minimal changes to the client code.
- **Decoupling:** The client code is decoupled from the concrete classes it uses, depending only on the product interface.

Conclusion

The Factory Design Pattern simplifies object creation by centralizing it within a factory. This pattern is ideal for scenarios where the exact type of object to create isn't known until runtime, or when the object creation involves complex logic.

Abstract Factory Pattern in C++

Overview

The Abstract Factory Pattern is a creational design pattern that provides an interface for creating families of related or dependent objects without specifying their concrete classes. This pattern is useful when you need to ensure that a set of related objects is used together.

Key Points:

- **Abstract Factory Pattern:** Creates families of related objects.
- **Factory Method Pattern:** Creates a single object but defers instantiation to subclasses.

Why Use Abstract Factory?

When you have multiple related objects (like buttons, checkboxes, etc.) that belong to different families (e.g., Windows, Mac), and you want to ensure that you use the objects from the same family together, the Abstract Factory Pattern is a good choice. This pattern helps to maintain consistency among products.

Example Scenario

Imagine you're designing a user interface (UI) toolkit that should work on different platforms like Windows and Mac. You want to create platform-specific UI elements like buttons and checkboxes, but you want to ensure that a Windows button is used with a Windows checkbox, and similarly for Mac.

Components of Abstract Factory Pattern

1. **Abstract Product:** Declares an interface for a type of product object. (e.g., `Button` , `Checkbox`)
2. **Concrete Product:** Implements the abstract product interface. (e.g., `WindowsButton` , `MacButton` , `WindowsCheckbox` , `MacCheckbox`)
3. **Abstract Factory:** Declares an interface for creating abstract products. (e.g., `GUIFactory`)
4. **Concrete Factory:** Implements the operations to create concrete product objects. (e.g., `WindowsFactory` , `MacFactory`)

Implementation

Here's a simple C++ implementation of the Abstract Factory Pattern:

```
#include <iostream>

// Abstract Product: Button
class Button {
public:
    virtual void render() const = 0;
    virtual ~Button() {}
};

// Abstract Product: Checkbox
class Checkbox {
public:
    virtual void render() const = 0;
    virtual ~Checkbox() {}
};

// Concrete Product: Windows Button
class WindowsButton : public Button {
public:
    void render() const override {
        std::cout << "Rendering Windows Button." << std::endl;
    }
};

// Concrete Product: Mac Button
class MacButton : public Button {
public:
    void render() const override {
        std::cout << "Rendering Mac Button." << std::endl;
    }
};

// Concrete Product: Windows Checkbox
class WindowsCheckbox : public Checkbox {
public:
    void render() const override {
        std::cout << "Rendering Windows Checkbox." << std::endl;
    }
};

// Concrete Product: Mac Checkbox
class MacCheckbox : public Checkbox {
public:
```

```

        void render() const override {
            std::cout << "Rendering Mac Checkbox." << std::endl;
        }
    };

// Abstract Factory
class GUIFactory {
public:
    virtual Button* createButton() const = 0;
    virtual Checkbox* createCheckbox() const = 0;
    virtual ~GUIFactory() {}
};

// Concrete Factory: Windows Factory
class WindowsFactory : public GUIFactory {
public:
    Button* createButton() const override {
        return new WindowsButton();
    }

    Checkbox* createCheckbox() const override {
        return new WindowsCheckbox();
    }
};

// Concrete Factory: Mac Factory
class MacFactory : public GUIFactory {
public:
    Button* createButton() const override {
        return new MacButton();
    }

    Checkbox* createCheckbox() const override {
        return new MacCheckbox();
    }
};

int main() {
    GUIFactory* factory;

    // Use a specific factory type
    factory = new WindowsFactory();
    Button* button = factory->createButton();
    Checkbox* checkbox = factory->createCheckbox();

    button->render();
    checkbox->render();

    delete button;
    delete checkbox;
    delete factory; // Clean up to avoid memory leaks

    factory = new MacFactory();
    button = factory->createButton();
    checkbox = factory->createCheckbox();

    button->render();
    checkbox->render();

    delete button;
    delete checkbox;
    delete factory; // Clean up to avoid memory leaks

    return 0;
}

```

Key Differences from Factory Method Pattern

- **Abstract Factory:** Focuses on creating **families** of related products. It ensures that a set of related objects are used together (e.g., Windows button with Windows checkbox).
- **Factory Method:** Focuses on creating a **single product**. It delegates the instantiation of a product to subclasses.

If you find a class overloaded with multiple Factory Methods—each responsible for creating different types of related objects—this might indicate that the class is taking on too many responsibilities, which can violate the Single Responsibility Principle (SRP).

The Single Responsibility Principle states that a class should have only one reason to change, meaning it should have only one responsibility. When a class starts to handle multiple factory methods, each creating different related objects, it's easy for the class to become complex and hard to maintain. This is where the Abstract Factory Pattern can come to the rescue.

When to Use Abstract Factory?

- When your system needs to be independent of how its objects are created.
- When you want to ensure that a family of related objects is used together.

- When you want to enforce consistency among products.

Benefits of Using Abstract Factory

- **Separation of Concerns:** Each factory (like WindowsFactory and MacFactory) is responsible for creating a family of related products. This separation keeps the code cleaner and each class focused on a single responsibility.
- **Scalability:** If you need to add a new platform, say LinuxFactory, you can do so without modifying the existing factories. This aligns with the Open/Closed Principle (OCP), where classes should be open for extension but closed for modification.
- **Maintainability:** By avoiding a single class with multiple factory methods, the code becomes easier to maintain. Each factory class is simple and focused on a specific set of related products.

Conclusion

The Abstract Factory Pattern is powerful when dealing with families of related objects, allowing for greater flexibility and consistency in object creation. By using this pattern, your code can be easily extended to support new families of products without modifying existing code.

For more information on other design patterns, check the corresponding README files in this directory.

Builder Design Pattern

Overview

The Builder Design Pattern is a creational pattern used to construct complex objects step by step. Unlike other creational patterns that create objects in a single step, the Builder Pattern allows you to build an object by specifying its various parts or properties one at a time. This approach is particularly useful when an object has many attributes or when different variations of the object need to be created.

When to Use the Builder Pattern?

- When the object to be created is complex and has multiple attributes.
- When you want to construct different representations of the same object.
- When you need to create an object step by step and ensure that all the necessary steps are performed.

Use the Builder pattern to get rid of a “telescoping constructor”.

Say you have a constructor with ten optional parameters. Calling such a beast is very inconvenient; therefore, you overload the constructor and create several shorter versions with fewer parameters. These constructors still refer to the main one, passing some default values into any omitted parameters.

```
class Pizza {
    Pizza(int size) { ... }
    Pizza(int size, boolean cheese) { ... }
    Pizza(int size, boolean cheese, boolean pepperoni) { ... }
    //.....
};
```

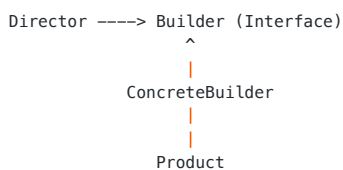
The Builder pattern lets you build objects step by step, using only those steps that you need.

Structure

The Builder Pattern involves the following key components:

1. **Builder (Interface):** Declares the building steps that are common to all types of builders. Each step typically returns the builder itself, allowing for method chaining.
2. **Concrete Builder:** Implements the building steps defined by the Builder interface. This is where the actual construction of the product happens.
3. **Product:** The complex object that is being built. The Builder sets its parts or attributes.
4. **Director (Optional):** An optional component that controls the construction process using a builder instance. It ensures that the construction steps are executed in a particular order.

UML Diagram



Example in C++

To fully understand the Builder Pattern, let's explore a more detailed example where we differentiate between basic and advanced features. We'll construct a `Car` object that can have both essential features (like engine and seats) and optional advanced features (like GPS, sunroof, and a high-end sound system).

Problem: Complex Object Construction

Imagine you are designing a `Car` class where the car can have a variety of configurations. Some cars might just need the basics (engine, seats), while others might need more luxurious options (GPS, sunroof). Using traditional constructors would lead to a telescoping constructor issue where you'd have constructors with an increasingly large number of parameters, making the code hard to read and maintain.

Solution: Builder Pattern

With the Builder Pattern, you can construct a `Car` step by step, making the process clear and flexible. We'll separate the construction of basic features and optional advanced features to show how the pattern can simplify object creation.

Step 1: Define the `Car` Product

First, define the `Car` class with the attributes that need to be set.


```

#include <iostream>
#include <string>

class Car {
private:
    std::string engine;
    int seats;
    bool hasGPS;
    bool hasSunroof;
    std::string soundSystem;

public:
    void setEngine(const std::string& engine) { this->engine = engine; }
    void setSeats(int seats) { this->seats = seats; }
    void setGPS(bool hasGPS) { this->hasGPS = hasGPS; }
    void setSunroof(bool hasSunroof) { this->hasSunroof = hasSunroof; }
    void setSoundSystem(const std::string& soundSystem) { this->soundSystem = soundSystem; }

    void show() const {
        std::cout << "Car with " << engine << " engine, " << seats << " seats";
        if (hasGPS) std::cout << ", GPS";
        if (hasSunroof) std::cout << ", sunroof";
        if (!soundSystem.empty()) std::cout << ", and " << soundSystem << " sound system";
        std::cout << "." << std::endl;
    }
};

```

Step 2: Define the Builder Interface

The `CarBuilder` interface declares methods for setting the car's attributes. We separate basic and advanced features.

```

class CarBuilder {
public:
    virtual void buildEngine() = 0;
    virtual void buildSeats() = 0;
    virtual void buildGPS() = 0;
    virtual void buildSunroof() = 0;
    virtual void buildSoundSystem() = 0;
    virtual Car* getCar() = 0;
    virtual ~CarBuilder() {}
};

```

Step 3: Implement the Concrete Builder

We'll create a concrete builder (`LuxuryCarBuilder`) that implements the `CarBuilder` interface. This builder can construct a basic car or add advanced features as needed.

```

class LuxuryCarBuilder : public CarBuilder {
private:
    Car* car;

public:
    LuxuryCarBuilder() { car = new Car(); }

    void buildEngine() override { car->setEngine("V8 Engine"); }
    void buildSeats() override { car->setSeats(4); }

    // Optional Features
    void buildGPS() override { car->setGPS(true); }
    void buildSunroof() override { car->setSunroof(true); }
    void buildSoundSystem() override { car->setSoundSystem("Bose Surround Sound"); }

    Car* getCar() override { return car; }
};

```

Step 4: Use a Director (Optional)

The `Director` class controls the construction process and can enforce specific sequences of building steps.

```

class CarDirector {
private:
    CarBuilder* builder;

public:
    void setBuilder(CarBuilder* builder) { this->builder = builder; }

    // Builds a basic car with just engine and seats
    Car* buildBasicCar() {
        builder->buildEngine();
        builder->buildSeats();
        return builder->getCar();
    }
};

```

```

    }

    // Builds a full-featured luxury car
    Car* buildLuxuryCar() {
        builder->buildEngine();
        builder->buildSeats();
        builder->buildGPS();
        builder->buildSunroof();
        builder->buildSoundSystem();
        return builder->getCar();
    }
};

```

Step 5: Client Code

Finally, let's use the `LuxuryCarBuilder` and `CarDirector` to create different configurations of the `Car`.

```

int main() {
    CarDirector director;
    LuxuryCarBuilder builder;

    director.setBuilder(&builder);

    // Build a basic car
    Car* basicCar = director.buildBasicCar();
    basicCar->show();
    delete basicCar;

    // Build a luxury car
    Car* luxuryCar = director.buildLuxuryCar();
    luxuryCar->show();
    delete luxuryCar;

    return 0;
}

```

How It Works

- **Basic Features:** The `buildEngine` and `buildSeats` methods are called to create the basic version of the `Car`.
- **Optional Features:** The `buildGPS`, `buildSunroof`, and `buildSoundSystem` methods add advanced features to the car. These methods can be skipped if you only need a basic version of the car.
- **Director:** The `CarDirector` simplifies the client code by encapsulating the construction logic, ensuring that the car is built in a valid and consistent manner.

Benefits of the Builder Pattern

- **Handles Telescopic Constructors:** The Builder Pattern avoids the problem of telescopic constructors by allowing you to add only the necessary parameters during construction.
- **Clear Separation:** There's a clear separation between the construction of basic and advanced features, making the code easier to understand and maintain.
- **Flexible Object Creation:** The same builder can be used to create different configurations of the product, like a basic or luxury car.
- **Improves Readability:** The step-by-step construction process is clear and easy to follow.
- **Flexible Object Creation:** You can create different variations of the object using the same construction process.
- **Encapsulation:** The construction details are hidden from the client, promoting encapsulation.

Conclusion

The Builder Pattern is an excellent solution when you need to construct complex objects with a mix of required and optional features. It keeps the code organized, makes object construction flexible, and eliminates the issues caused by telescopic constructors.

Adapter Design Pattern in C++

Introduction

The Adapter Pattern is a structural design pattern that allows objects with incompatible interfaces to collaborate. It acts as a bridge between two incompatible interfaces by wrapping an existing class with a new interface, making it compatible with another class.

Use Case

Imagine you have a class that expects a certain interface, but you have another class that provides the functionality you need but with a different interface. Instead of modifying the existing classes (which might not be possible), you create an Adapter class that makes the two classes work together.

Structure

The Adapter Pattern typically involves the following components:

1. **Target Interface:** The interface that the client expects to work with.
2. **Adaptee:** The existing class that needs to be adapted to the Target Interface.
3. **Adapter:** A class that implements the Target Interface and translates the requests from the client to the Adaptee.

Example

Let's consider an example where you have an existing media player that only supports playing MP3 files, but you want to play MP4 and VLC files without changing the existing media player code.

```
#include <iostream>
#include <string>

// Target Interface
class MediaPlayer {
public:
    virtual void play(const std::string& audioType, const std::string& fileName) const = 0;
    virtual ~MediaPlayer() {}
};

// Adaptee Interface
class AdvancedMediaPlayer {
public:
    virtual void playMp4(const std::string& fileName) const = 0;
    virtual void playVlc(const std::string& fileName) const = 0;
    virtual ~AdvancedMediaPlayer() {}
};

// Concrete Adaptee: Mp4Player
class Mp4Player : public AdvancedMediaPlayer {
public:
    void playMp4(const std::string& fileName) const override {
        std::cout << "Playing mp4 file. Name: " << fileName << std::endl;
    }

    void playVlc(const std::string& fileName) const override {
        // Do nothing
    }
};

// Concrete Adaptee: VlcPlayer
class VlcPlayer : public AdvancedMediaPlayer {
public:
    void playMp4(const std::string& fileName) const override {
        // Do nothing
    }

    void playVlc(const std::string& fileName) const override {
        std::cout << "Playing vlc file. Name: " << fileName << std::endl;
    }
};

// Concrete Adaptee: Mp3Player
class Mp3Player : public MediaPlayer {
public:
    void play(const std::string& audioType, const std::string& fileName) const override {
        if (audioType == "mp3") {
            std::cout << "Playing mp3 file. Name: " << fileName << std::endl;
        }
    }
};
```

```
// Adapter Class
class MediaAdapter : public MediaPlayer {
private:
    AdvancedMediaPlayer* advancedPlayer;

public:
    MediaAdapter(const std::string& audioType) {
        if (audioType == "mp4") {
            advancedPlayer = new Mp4Player();
        } else if (audioType == "vlc") {
            advancedPlayer = new VlcPlayer();
        } else {
            advancedPlayer = nullptr;
        }
    }

    void play(const std::string& audioType, const std::string& fileName) const override {
        if (audioType == "mp4") {
            advancedPlayer->playMp4(fileName);
        } else if (audioType == "vlc") {
            advancedPlayer->playVlc(fileName);
        }
    }

    ~MediaAdapter() {
        delete advancedPlayer;
    }
};

// Client: AudioPlayer
class AudioPlayer : public MediaPlayer {
private:
    MediaAdapter* mediaAdapter;
    Mp3Player* mp3Player;

public:
    AudioPlayer() : mediaAdapter(nullptr), mp3Player(new Mp3Player()) {}

    void play(const std::string& audioType, const std::string& fileName) const override {
        if (audioType == "mp3") {
            mp3Player->play(audioType, fileName);
        } else if (audioType == "mp4" || audioType == "vlc") {
            mediaAdapter = new MediaAdapter(audioType);
            mediaAdapter->play(audioType, fileName);
            delete mediaAdapter;
        } else {
            std::cout << "Invalid media. " << audioType << " format not supported." << std::endl;
        }
    }

    ~AudioPlayer() {
        delete mp3Player;
    }
};

// Main function to demonstrate the Adapter Pattern
int main() {
    AudioPlayer player;
    player.play("mp3", "song.mp3");
    player.play("mp4", "video.mp4");
    player.play("vlc", "movie.vlc");
    player.play("avi", "file.avi");

    return 0;
}
```

Benefits of Adapter Pattern

1. **Flexibility:** Allows you to integrate classes that couldn't otherwise work together.
2. **Reusability:** You can reuse existing classes with different interfaces without altering their code.
3. **Single Responsibility:** The Adapter class focuses on resolving interface mismatches, keeping other classes free of this concern.

How to Identify the Adapter Pattern

You can identify the Adapter Pattern in code when:

- You see a class (Adapter) that implements an interface expected by a client (Target Interface), but internally uses another class (Adaptee) to perform the actual work.
- The Adapter class is used to "adapt" an existing class with an incompatible interface to match the required interface.
- There are often multiple incompatible classes (e.g., Mp4Player , VlcPlayer), and the adapter can wrap these classes to make them compatible with a single client.

Signs of Adapter Pattern in Code

- The presence of a class that implements a known interface but internally delegates calls to a different class.
- Code where existing classes or APIs are being adapted to a new interface without modifying the original classes.
- The use of composition over inheritance, where the adapter holds a reference to the adaptee.

Conclusion

The Adapter Pattern is a powerful tool in your design pattern toolkit, allowing you to integrate and reuse classes with incompatible interfaces. By identifying the need for an Adapter, you can avoid modifying existing classes and instead wrap them with a class that translates the interface, keeping your code flexible and maintainable.

Composite Design Pattern in C++

The Composite Pattern is a structural design pattern used to represent part-whole hierarchies. This pattern allows individual objects and groups of objects to be treated uniformly.

Overview

In the Composite Pattern:

- **Component Interface:** Defines a common interface for all objects in the composition.
- **Leaf:** Represents a simple object in the hierarchy (e.g., a file).
- **Composite:** Represents a container (e.g., a directory) that can hold other components.

Example: File System

We will use the Composite Pattern to implement a file system where:

- A `File` is a leaf node.
- A `Directory` is a composite node that can contain both files and other directories.

Implementation in C++

1. Define the Component Interface

The `FileSystemEntity` interface defines the common behavior for both `File` and `Directory`.

```
#include <iostream>
#include <vector>
#include <memory>
#include <string>
#include <algorithm>

class FileSystemEntity {
public:
    virtual void display(int indent = 0) const = 0;
    virtual ~FileSystemEntity() = default;
};
```

2. Implement the Leaf Class: File

A `File` is a basic entity that implements the `FileSystemEntity` interface.

```
class File : public FileSystemEntity {
private:
    std::string name;

public:
    explicit File(const std::string& fileName) : name(fileName) {}

    void display(int indent = 0) const override {
        std::cout << std::string(indent, ' ') << "File: " << name << std::endl;
    }
};
```

3. Implement the Composite Class: Directory

A `Directory` is a composite object that contains other `FileSystemEntity` objects.

```
class Directory : public FileSystemEntity {
private:
    std::string name;
    std::vector<std::shared_ptr<FileSystemEntity>> children;

public:
    explicit Directory(const std::string& dirName) : name(dirName) {}

    void add(const std::shared_ptr<FileSystemEntity>& entity) {
        children.push_back(entity);
    }

    void remove(const std::shared_ptr<FileSystemEntity>& entity) {
        children.erase(
            std::remove(children.begin(), children.end(), entity),
            children.end()
        );
    }
};
```

```

    }

    void display(int indent = 0) const override {
        std::cout << std::string(indent, ' ') << "Directory: " << name << std::endl;
        for (const auto& child : children) {
            child->display(indent + 2);
        }
    }
}

};

```

4. Client Code: Building the File System

The client code demonstrates how to use the `File` and `Directory` classes to build and display a file system.

```

int main() {
    // Create individual files
    auto file1 = std::make_shared<File>("File1.txt");
    auto file2 = std::make_shared<File>("File2.txt");
    auto file3 = std::make_shared<File>("File3.txt");

    // Create directories
    auto dir1 = std::make_shared<Directory>("Dir1");
    auto dir2 = std::make_shared<Directory>("Dir2");
    auto root = std::make_shared<Directory>("Root");

    // Build the hierarchy
    dir1->add(file1);
    dir1->add(file2);
    dir2->add(file3);
    root->add(dir1);
    root->add(dir2);

    // Display the structure
    root->display();

    return 0;
}

```

5. Example Output

When the above code is executed, the following output is produced:

```

Directory: Root
Directory: Dir1
File: File1.txt
File: File2.txt
Directory: Dir2
File: File3.txt

```

Advantages

1. **Uniformity:** Treats individual and composite objects uniformly.
2. **Scalability:** Easily add new types of components without changing existing code.
3. **Hierarchy Representation:** Ideal for tree-like structures.

Disadvantages

1. **Complexity:** Recursive structures can make debugging and maintenance harder.
2. **Overhead:** Storing and traversing large trees may consume significant resources.

Use Cases

- File systems (e.g., files and directories).
- Organization charts (e.g., employees and managers).
- GUI frameworks (e.g., windows and widgets).
- Graphics systems (e.g., shapes composed of lines and circles).

Conclusion

The Composite Pattern simplifies working with hierarchical data structures by enabling the uniform treatment of individual and composite objects. It is widely used in scenarios requiring tree-like structures.

Decorator Design Pattern in C++

The Decorator Pattern is a structural design pattern used to dynamically add new behavior to objects without altering their structure. This pattern adheres to the Open/Closed Principle by allowing functionality to be extended without modifying existing code.

Overview

In the Decorator Pattern:

- **Component Interface:** Defines the interface for objects that can have responsibilities added to them.
- **Concrete Component:** The base implementation of the component interface.
- **Decorator:** A class that wraps a component and adds additional behavior.
- **Concrete Decorators:** Implement specific enhancements to the component.

Example: Notification System

We will use the Decorator Pattern to implement a notification system where:

- A `Notifier` interface defines the common behavior.
- A `BasicNotifier` sends basic notifications.
- Decorators add additional notification methods (e.g., SMS, Email, Push notifications).

Implementation in C++

Here is the complete code for the Decorator Pattern:

```
#include <iostream>
#include <memory>
#include <string>

// Component Interface
class Notifier {
public:
    virtual void send(const std::string& message) const = 0;
    virtual ~Notifier() = default;
};

// Concrete Component
class BasicNotifier : public Notifier {
public:
    void send(const std::string& message) const override {
        std::cout << "Basic Notification: " << message << std::endl;
    }
};

// Decorator Base Class
class NotifierDecorator : public Notifier {
protected:
    std::shared_ptr<Notifier> wrappedNotifier;

public:
    explicit NotifierDecorator(std::shared_ptr<Notifier> notifier) : wrappedNotifier(std::move(notifier)) {}
    void send(const std::string& message) const override {
        wrappedNotifier->send(message);
    }
};

// Concrete Decorators
class SMSNotifier : public NotifierDecorator {
public:
    explicit SMSNotifier(std::shared_ptr<Notifier> notifier) : NotifierDecorator(std::move(notifier)) {}

    void send(const std::string& message) const override {
        NotifierDecorator::send(message);
        std::cout << "SMS Notification: " << message << std::endl;
    }
};

class EmailNotifier : public NotifierDecorator {
public:
    explicit EmailNotifier(std::shared_ptr<Notifier> notifier) : NotifierDecorator(std::move(notifier)) {}

    void send(const std::string& message) const override {
        NotifierDecorator::send(message);
        std::cout << "Email Notification: " << message << std::endl;
    }
};

class PushNotifier : public NotifierDecorator {
```



```

public:
    explicit PushNotifier(std::shared_ptr<Notifier> notifier) : NotifierDecorator(std::move(notifier)) {}

    void send(const std::string& message) const override {
        NotifierDecorator::send(message);
        std::cout << "Push Notification: " << message << std::endl;
    }
};

int main() {
    // Create a basic notifier
    auto basicNotifier = std::make_shared<BasicNotifier>();

    // Decorate with SMS notifications
    auto smsNotifier = std::make_shared<SMSNotifier>(basicNotifier);

    // Decorate with Email notifications
    auto emailNotifier = std::make_shared<EmailNotifier>(smsNotifier);

    // Decorate with Push notifications
    auto pushNotifier = std::make_shared<PushNotifier>(emailNotifier);

    // Send a notification
    pushNotifier->send("Hello, World!");

    return 0;
}

```

Example Output

When the above code is executed, the following output is produced:

```

Basic Notification: Hello, World!
SMS Notification: Hello, World!
Email Notification: Hello, World!
Push Notification: Hello, World!

```

Advantages

1. **Flexibility:** Add new behavior without modifying existing code.
2. **Reusable Decorators:** Decorators can be reused across multiple objects.
3. **Open/Closed Principle:** Extend functionality without altering base classes.

Disadvantages

1. **Complexity:** May introduce a large number of small classes.
2. **Overhead:** Wrapping objects can add overhead.

Use Cases

- Adding cross-cutting concerns like logging, authentication, or caching.
- Extending UI components in GUI frameworks.
- Enhancing behaviors of objects in a modular way.

Conclusion

The Decorator Pattern provides a flexible and modular way to extend object behavior without altering the underlying class. It is particularly useful when dealing with cross-cutting concerns or dynamically changing responsibilities.

Facade Design Pattern in C++

The Facade Pattern is a structural design pattern that provides a simplified interface to a larger body of code, such as a complex subsystem. It aims to make a system easier to use by masking its complexity behind a single, unified interface.

Overview

In the Facade Pattern:

- **Facade:** Provides a simple interface to the subsystem and delegates client requests to the appropriate subsystem classes.
- **Subsystems:** Represent the complex system, which is often a set of interrelated classes.

Example: Home Theater System

We will use the Facade Pattern to control a home theater system with components such as a projector, sound system, and DVD player. The `HomeTheaterFacade` class will provide a simplified interface to operate the entire system.

Implementation in C++

Here is the complete code for the Facade Pattern:

```
#include <iostream>
#include <string>

// Subsystem: Projector
class Projector {
public:
    void on() const {
        std::cout << "Projector is now ON." << std::endl;
    }
    void off() const {
        std::cout << "Projector is now OFF." << std::endl;
    }
    void setWideScreenMode() const {
        std::cout << "Projector is set to Widescreen Mode." << std::endl;
    }
};

// Subsystem: Sound System
class SoundSystem {
public:
    void on() const {
        std::cout << "Sound System is now ON." << std::endl;
    }
    void off() const {
        std::cout << "Sound System is now OFF." << std::endl;
    }
    void setVolume(int level) const {
        std::cout << "Sound System volume set to " << level << "." << std::endl;
    }
};

// Subsystem: DVD Player
class DVDPlayer {
public:
    void on() const {
        std::cout << "DVD Player is now ON." << std::endl;
    }
    void off() const {
        std::cout << "DVD Player is now OFF." << std::endl;
    }
    void play(const std::string& movie) const {
        std::cout << "Playing movie: " << movie << std::endl;
    }
};

// Facade: Home Theater
class HomeTheaterFacade {
private:
    Projector projector;
    SoundSystem soundSystem;
    DVDPlayer dvdPlayer;

public:
    void watchMovie(const std::string& movie) {
        std::cout << "\nPreparing to watch a movie..." << std::endl;
        projector.on();
        projector.setWideScreenMode();
        soundSystem.on();
        soundSystem.setVolume(50);
    }
};
```

```

        dvdPlayer.on();
        dvdPlayer.play(movie);
        std::cout << "Enjoy your movie!\n" << std::endl;
    }

    void endMovie() {
        std::cout << "\nShutting down the home theater system..." << std::endl;
        dvdPlayer.off();
        soundSystem.off();
        projector.off();
        std::cout << "Goodbye!\n" << std::endl;
    }
};

int main() {
    // Create the facade
    HomeTheaterFacade homeTheater;

    // Use the facade to control the system
    homeTheater.watchMovie("Inception");
    homeTheater.endMovie();

    return 0;
}

```

Example Output

When the above code is executed, the following output is produced:

```

Preparing to watch a movie...
Projector is now ON.
Projector is set to Widescreen Mode.
Sound System is now ON.
Sound System volume set to 50.
DVD Player is now ON.
Playing movie: Inception
Enjoy your movie!

Shutting down the home theater system...
DVD Player is now OFF.
Sound System is now OFF.
Projector is now OFF.
Goodbye!

```

Advantages

1. **Simplified Interface:** Makes complex subsystems easier to use.
2. **Decoupling:** Shields clients from the details of the subsystem.
3. **Flexibility:** Allows subsystems to evolve independently of their clients.

Disadvantages

1. **Limited Control:** May restrict the advanced features of the subsystem.
2. **Dependency:** The facade class becomes a single point of modification.

Use Cases

- Complex systems requiring a simple and consistent interface.
- Libraries with intricate APIs that need to be hidden from the user.
- Applications where subsystems may change without affecting the client.

Conclusion

The Facade Pattern provides a convenient interface to a complex system, improving usability and reducing dependencies. It is widely used in software libraries and frameworks to simplify interactions with their components.

Flyweight Design Pattern in C++

The Flyweight Pattern is a structural design pattern that reduces memory usage by sharing common data among multiple objects. It separates intrinsic (shared) and extrinsic (unique) data, allowing a large number of fine-grained objects to be managed efficiently.

Overview

In the Flyweight Pattern:

- **Flyweight Interface:** Defines a common interface for shared objects.
- **Concrete Flyweight:** Implements the Flyweight interface and shares data.
- **Flyweight Factory:** Manages and provides shared Flyweight objects.
- **Client:** Combines shared Flyweight objects with unique extrinsic states.

Example: Text Formatting

We will use the Flyweight Pattern to render text with shared character formatting (intrinsic state) and unique positioning (extrinsic state).

Implementation in C++

Here is the complete code for the Flyweight Pattern:

```
#include <iostream>
#include <unordered_map>
#include <memory>
#include <string>

// Flyweight Interface
class Character {
public:
    virtual void display(int positionX, int positionY) const = 0;
    virtual ~Character() = default;
};

// Concrete Flyweight: Shared Character Data
class ConcreteCharacter : public Character {
private:
    char symbol;
    std::string font;

public:
    ConcreteCharacter(char sym, const std::string& f) : symbol(sym), font(f) {}

    void display(int positionX, int positionY) const override {
        std::cout << "Character: " << symbol
                  << ", Font: " << font
                  << ", Position: (" << positionX << ", " << positionY << ")" << std::endl;
    }
};

// Flyweight Factory
class CharacterFactory {
private:
    std::unordered_map<char, std::shared_ptr<Character>> characters;

public:
    std::shared_ptr<Character> getCharacter(char symbol, const std::string& font) {
        auto it = characters.find(symbol);
        if (it == characters.end()) {
            auto character = std::make_shared<ConcreteCharacter>(symbol, font);
            characters[symbol] = character;
            return character;
        }
        return it->second;
    }
};

// Client Code
int main() {
    CharacterFactory factory;

    // Shared Characters
    auto charA = factory.getCharacter('A', "Arial");
    auto charB = factory.getCharacter('B', "Arial");
    auto charARepet = factory.getCharacter('A', "Arial"); // Reuses the existing 'A'

    // Display characters at different positions
    charA->display(10, 20);
    charB->display(15, 25);
    charARepet->display(30, 40); // Reuses the same intrinsic data for 'A'
```

```
}    return 0;  
}
```

Example Output

When the above code is executed, the following output is produced:

```
Character: A, Font: Arial, Position: (10, 20)  
Character: B, Font: Arial, Position: (15, 25)  
Character: A, Font: Arial, Position: (30, 40)
```

Advantages

1. **Reduced Memory Usage:** Minimizes duplication of intrinsic state.
2. **Efficient Object Management:** Ideal for systems with many similar objects.

Disadvantages

1. **Complexity:** Increases code complexity due to state separation.
2. **Limited Flexibility:** Requires careful design to separate intrinsic and extrinsic data.

Use Cases

- Text editors (e.g., shared glyph data for characters).
- Graphics systems (e.g., shared object models in 3D rendering).
- Game development (e.g., managing many similar entities like trees or enemies).

Conclusion

The Flyweight Pattern is a powerful tool for optimizing memory usage in applications with many similar objects. By sharing intrinsic data and managing unique extrinsic state, it strikes a balance between efficiency and functionality.

Proxy Design Pattern in C++

The Proxy Pattern is a structural design pattern that provides a surrogate or placeholder for another object to control access to it. It allows adding additional behavior or managing access to the real object without modifying it directly.

Overview

In the Proxy Pattern:

- **Subject Interface:** Defines the common interface for RealSubject and Proxy.
- **RealSubject:** The actual object that performs the core operations.
- **Proxy:** Acts as a placeholder for the RealSubject, adding functionality like access control, logging, or lazy initialization.

Example: Image Viewer

We will implement an image viewer where:

- A `RealImage` is the actual image being displayed.
- A `ProxyImage` acts as a proxy for `RealImage`, loading it lazily and adding logging functionality.

Implementation in C++

Here is the complete code for the Proxy Pattern:

```
#include <iostream>
#include <string>
#include <memory>

// Subject Interface
class Image {
public:
    virtual void display() const = 0;
    virtual ~Image() = default;
};

// RealSubject
class RealImage : public Image {
private:
    std::string fileName;

    void loadFromDisk() const {
        std::cout << "Loading image from disk: " << fileName << std::endl;
    }

public:
    explicit RealImage(const std::string& file) : fileName(file) {
        loadFromDisk();
    }

    void display() const override {
        std::cout << "Displaying image: " << fileName << std::endl;
    }
};

// Proxy
class ProxyImage : public Image {
private:
    std::string fileName;
    mutable std::shared_ptr<RealImage> realImage; // Lazy initialization

public:
    explicit ProxyImage(const std::string& file) : fileName(file), realImage(nullptr) {}

    void display() const override {
        if (!realImage) {
            realImage = std::make_shared<RealImage>(fileName);
        }
        realImage->display();
    }
};

int main() {
    // Use ProxyImage to access RealImage
    ProxyImage proxyImage("test_image.jpg");

    // The real image is loaded only when display is called
    std::cout << "First call to display:" << std::endl;
    proxyImage.display();

    std::cout << "\nSecond call to display:" << std::endl;
```

```
    proxyImage.display();  
  
    return 0;  
}
```

Example Output

When the above code is executed, the following output is produced:

```
First call to display:  
Loading image from disk: test_image.jpg  
Displaying image: test_image.jpg  
  
Second call to display:  
Displaying image: test_image.jpg
```

Advantages

1. **Lazy Initialization**: The real object is created only when needed, saving resources.
2. **Access Control**: Restrict or control access to the real object.
3. **Additional Behavior**: Add logging, caching, or other functionality transparently.

Disadvantages

1. **Complexity**: Adds an extra layer of indirection, which can make debugging harder.
2. **Performance Overhead**: Proxy adds overhead, especially if the real object is lightweight.

Use Cases

- **Virtual Proxy**: Delay the creation and initialization of expensive objects.
- **Remote Proxy**: Represent an object in a different address space (e.g., network communication).
- **Protection Proxy**: Control access to sensitive objects based on permissions.
- **Logging Proxy**: Add logging or tracking functionality.

Conclusion

The Proxy Pattern provides a flexible way to manage access to an object while keeping its interface consistent. It is particularly useful for scenarios requiring lazy initialization, logging, or access control.

Chain of Responsibility Design Pattern in C++

The Chain of Responsibility Pattern is a behavioral design pattern that allows a request to be passed along a chain of handlers until one of them handles it. This pattern decouples the sender of the request from its receivers.

Overview

In the Chain of Responsibility Pattern:

- **Handler Interface:** Defines the interface for handling requests and setting the next handler.
- **Concrete Handlers:** Implement the handler interface and decide whether to process a request or pass it to the next handler.
- **Client:** Initiates requests to be handled by the chain.

Example: Customer Support System

We will implement a customer support system where requests are passed along a chain of support levels (e.g., Low-Level Support, Mid-Level Support, and High-Level Support).

Implementation in C++

Here is the complete code for the Chain of Responsibility Pattern:

```
#include <iostream>
#include <memory>
#include <string>

// Handler Interface
class SupportHandler {
protected:
    std::shared_ptr<SupportHandler> nextHandler;

public:
    void setNextHandler(const std::shared_ptr<SupportHandler>& handler) {
        nextHandler = handler;
    }

    virtual void handleRequest(const std::string& issue, int severity) = 0;
    virtual ~SupportHandler() = default;
};

// Concrete Handler: Low-Level Support
class LowLevelSupport : public SupportHandler {
public:
    void handleRequest(const std::string& issue, int severity) override {
        if (severity <= 2) {
            std::cout << "Low-Level Support: Handling issue - " << issue << std::endl;
        } else if (nextHandler) {
            nextHandler->handleRequest(issue, severity);
        } else {
            std::cout << "Low-Level Support: Unable to handle issue." << std::endl;
        }
    }
};

// Concrete Handler: Mid-Level Support
class MidLevelSupport : public SupportHandler {
public:
    void handleRequest(const std::string& issue, int severity) override {
        if (severity <= 5) {
            std::cout << "Mid-Level Support: Handling issue - " << issue << std::endl;
        } else if (nextHandler) {
            nextHandler->handleRequest(issue, severity);
        } else {
            std::cout << "Mid-Level Support: Unable to handle issue." << std::endl;
        }
    }
};

// Concrete Handler: High-Level Support
class HighLevelSupport : public SupportHandler {
public:
    void handleRequest(const std::string& issue, int severity) override {
        if (severity <= 10) {
            std::cout << "High-Level Support: Handling issue - " << issue << std::endl;
        } else {
            std::cout << "High-Level Support: Unable to handle issue. Escalating further." << std::endl;
        }
    }
};
```



```
// Client Code
int main() {
    // Create support handlers
    auto lowLevel = std::make_shared<LowLevelSupport>();
    auto midLevel = std::make_shared<MidLevelSupport>();
    auto highLevel = std::make_shared<HighLevelSupport>();

    // Set up the chain of responsibility
    lowLevel->setNextHandler(midLevel);
    midLevel->setNextHandler(highLevel);

    // Client sends requests
    lowLevel->handleRequest("Password reset", 1); // Handled by Low-Level Support
    lowLevel->handleRequest("System crash", 4);   // Handled by Mid-Level Support
    lowLevel->handleRequest("Data breach", 9);     // Handled by High-Level Support
    lowLevel->handleRequest("Critical failure", 11); // Not handled

    return 0;
}
```

Example Output

When the above code is executed, the following output is produced:

```
Low-Level Support: Handling issue - Password reset
Mid-Level Support: Handling issue - System crash
High-Level Support: Handling issue - Data breach
High-Level Support: Unable to handle issue. Escalating further.
```

Advantages

1. **Decoupling:** Decouples the sender of a request from its receivers.
2. **Flexibility:** Easily add or reorder handlers in the chain.
3. **Responsibility Sharing:** Distributes the handling of requests among multiple objects.

Disadvantages

1. **Uncertainty:** No guarantee that a request will be handled.
2. **Debugging:** Can make debugging harder due to dynamic behavior.

Use Cases

- Logging frameworks with different levels (e.g., debug, info, error).
- Customer support systems with multiple support levels.
- Event handling in GUI frameworks.

Conclusion

The Chain of Responsibility Pattern provides a flexible and scalable approach to handle requests by passing them along a chain of handlers. It is particularly useful in systems requiring dynamic and decoupled request handling.

Iterator Design Pattern in C++

The Iterator Pattern is a behavioral design pattern that provides a way to sequentially access elements of a collection without exposing its underlying representation.

Overview

In the Iterator Pattern:

- **Iterator Interface:** Defines methods to traverse through elements.
- **Concrete Iterator:** Implements the iterator interface for a specific collection.
- **Aggregate Interface:** Defines a method to create an iterator.
- **Concrete Aggregate:** Implements the aggregate interface and holds the collection.
- **Client:** Uses the iterator to access elements.

Example: Book Collection

We will implement a book collection where an iterator is used to traverse through the list of books.

Implementation in C++

Here is the complete code for the Iterator Pattern:

```
#include <iostream>
#include <vector>
#include <memory>
#include <string>

// Iterator Interface
template <typename T>
class Iterator {
public:
    virtual bool hasNext() const = 0;
    virtual T next() = 0;
    virtual ~Iterator() = default;
};

// Concrete Iterator
class BookIterator : public Iterator<std::string> {
private:
    const std::vector<std::string>& books;
    size_t index;

public:
    explicit BookIterator(const std::vector<std::string>& bookCollection)
        : books(bookCollection), index(0) {}

    bool hasNext() const override {
        return index < books.size();
    }

    std::string next() override {
        if (!hasNext()) {
            throw std::out_of_range("No more elements.");
        }
        return books[index++];
    }
};

// Aggregate Interface
class BookCollection {
public:
    virtual std::shared_ptr<Iterator<std::string>> createIterator() const = 0;
    virtual ~BookCollection() = default;
};

// Concrete Aggregate
class Library : public BookCollection {
private:
    std::vector<std::string> books;

public:
    void addBook(const std::string& book) {
        books.push_back(book);
    }

    std::shared_ptr<Iterator<std::string>> createIterator() const override {
        return std::make_shared<BookIterator>(books);
    }
};
```

```
// Client Code
int main() {
    // Create a library and add books
    Library library;
    library.addBook("The Catcher in the Rye");
    library.addBook("To Kill a Mockingbird");
    library.addBook("1984");
    library.addBook("Moby-Dick");

    // Get an iterator for the library
    auto iterator = library.createIterator();

    // Traverse the book collection
    std::cout << "Books in the library:" << std::endl;
    while (iterator->hasNext()) {
        std::cout << "- " << iterator->next() << std::endl;
    }

    return 0;
}
```

Example Output

When the above code is executed, the following output is produced:

```
Books in the library:
- The Catcher in the Rye
- To Kill a Mockingbird
- 1984
- Moby-Dick
```

Advantages

1. **Encapsulation:** The underlying structure of the collection is hidden from the client.
2. **Flexibility:** Iterators can work with different types of collections.
3. **Reusability:** Iterator logic is reusable across various client applications.

Disadvantages

1. **Overhead:** Creating and managing iterator objects may add complexity.
2. **Single Responsibility:** Adding an iterator might violate the Single Responsibility Principle for simple collections.

Use Cases

- Iterating over a collection of objects.
- Implementing traversal algorithms in tree or graph structures.
- Standard Template Library (STL) containers in C++.

Conclusion

The Iterator Pattern provides a robust mechanism for accessing elements in a collection while abstracting away the underlying details of the collection's structure. It is widely used in scenarios requiring sequential traversal.

Practice Questions to Test Your Design Skills

Welcome to the part where you implement what you learned!

These fun and challenging scenarios are crafted to help you flex your design muscles. By solving these real-world problems, you'll get hands-on experience with design patterns and strengthen your problem-solving abilities.

How to Get Started

1. Read the scenario and understand the requirements.
2. Think about the best design patterns to address the challenges.
3. Fire up your editor and start coding in your favorite language (C++ is a great choice!).
4. Verify your solution and see if it ticks all the boxes.

A Quick Note

You don't need to build the entire system for these scenarios. Instead, focus on creating a thoughtful class design that outlines the structure and relationships between key components. This will help you practice designing robust and scalable architectures.

Ready? Let's dive in!

Scenario 1: Design a Library Management System

Imagine you're creating a system for a library. The system should allow users to search for books by title, author, or genre, and borrow or return books. Librarians should be able to add new books and manage inventory.

What You'll Need to Do:

- Implement book search functionality.
- Manage book borrowing and returning.
- Allow librarians to add and remove books.

Hint: Use **Factory** for book creation and **Observer** to manage inventory updates.

Scenario 2: Develop a Ticket Booking System

Design a system where users can search for and book tickets for movies, concerts, or flights. Users should also have the ability to cancel tickets and view their booking history.

Your To-Do List:

- Implement ticket search and booking.
- Handle cancellations and booking history.
- Ensure real-time availability updates.

Hint: Use **Singleton** for availability management and **State** for ticket statuses.

Scenario 3: Build a Weather Notification App

Imagine an app that provides real-time weather updates to users. Users can subscribe to notifications based on their preferences, such as rain alerts, temperature changes, or storm warnings.

Your Challenges:

- Allow users to subscribe to specific types of notifications.
- Send real-time updates to subscribers.
- Support multiple notification types.

Hint: **Observer** pattern is perfect for managing subscriptions.

Scenario 4: Create a Personal Finance Tracker

Design a system to help users manage their finances. The app should let users track income and expenses, categorize transactions, and generate monthly reports.

What You'll Build:

- Transaction tracking and categorization.
- Monthly financial summary reports.
- Notifications for budget limits.

Hint: Use **Builder** for report generation and **Decorator** for adding notifications.

Scenario 5: Develop a Chat Application

Create a simple chat application where users can send messages to each other. The system should support one-on-one and group chats, message delivery status, and notifications.

What You'll Work On:

- Enable one-on-one and group chats.
- Show message delivery and read statuses.
- Implement notifications for new messages.

Hint: Use **Mediator** for managing chat sessions and **Observer** for notifications.

Important Tip

For each scenario:

1. Analyze the problem and break it down into clear requirements.
2. Identify the design pattern(s) that best address the challenges.
3. Explain your reasoning for choosing the pattern(s) with real-world relevance.
4. Implement your solution step-by-step, ensuring it meets the requirements.
5. Test your design to confirm it works as intended.

Enjoy the process, and remember—great design takes practice. You've got this!